```
addi $t0, $zero, 3 | operator reg, reg, immediate
addi $t1, $zero, 5 | operator reg, reg, immediate
add  $t2, $t0, $t1 | operator reg, reg, reg, space, operator
sub  $t3, $t1, $t0 | operator reg, reg, reg, space, operator
```

001000 00000 01000 0000000000000011
001000 00000 01001 0000000000000101
000000 01000 01001 01010 00000 100000
000000 01001 01000 01011 00000 100010

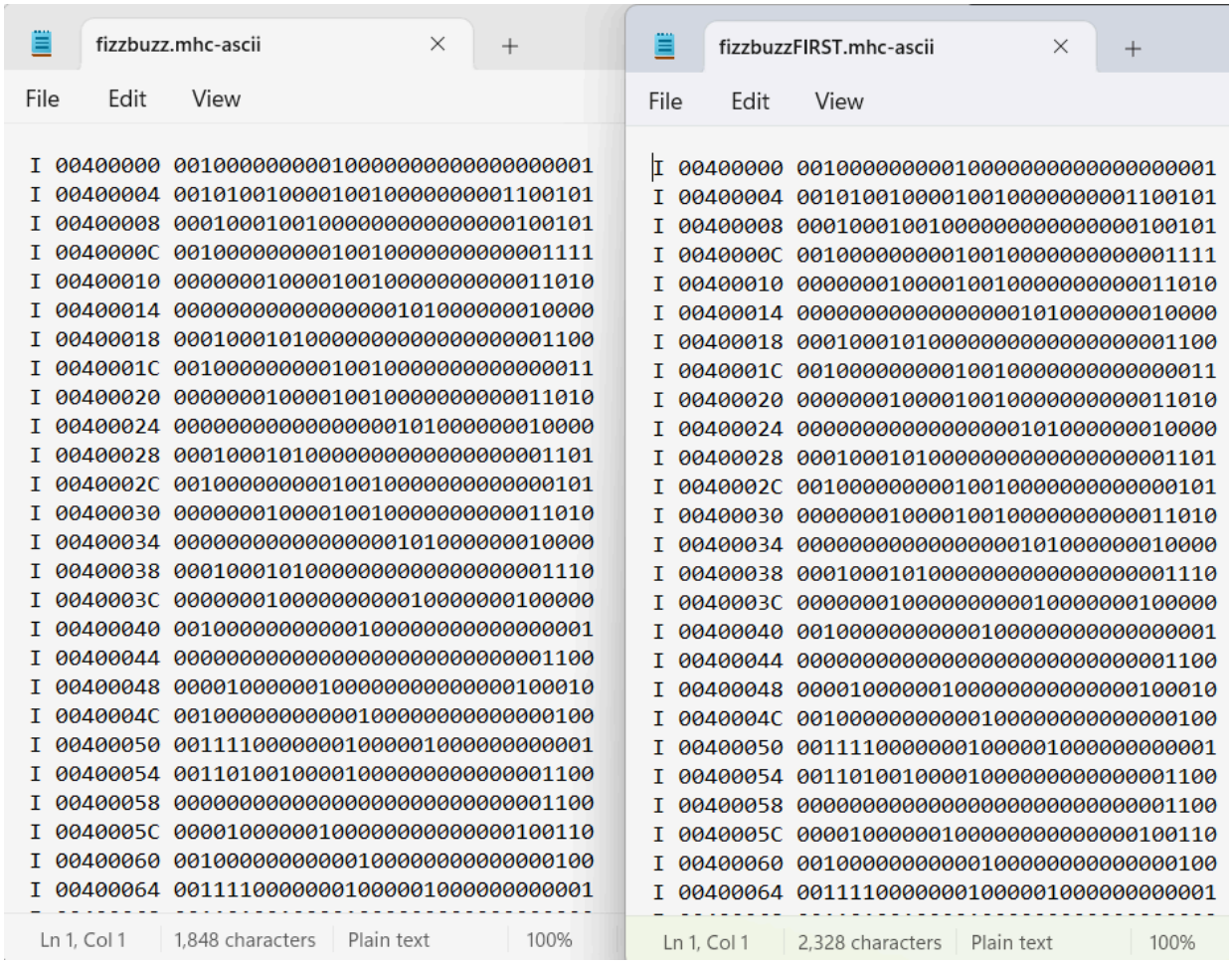
00100000 00001000 00000000 00000011 | 20 08 00 03
00100000 00001001 00000000 00000101 | 20 09 00 05
00000001 00001001 01010000 00100000 | 01 09 05 02
00000001 00101000 01011000 00100010 | 01 28 58 22

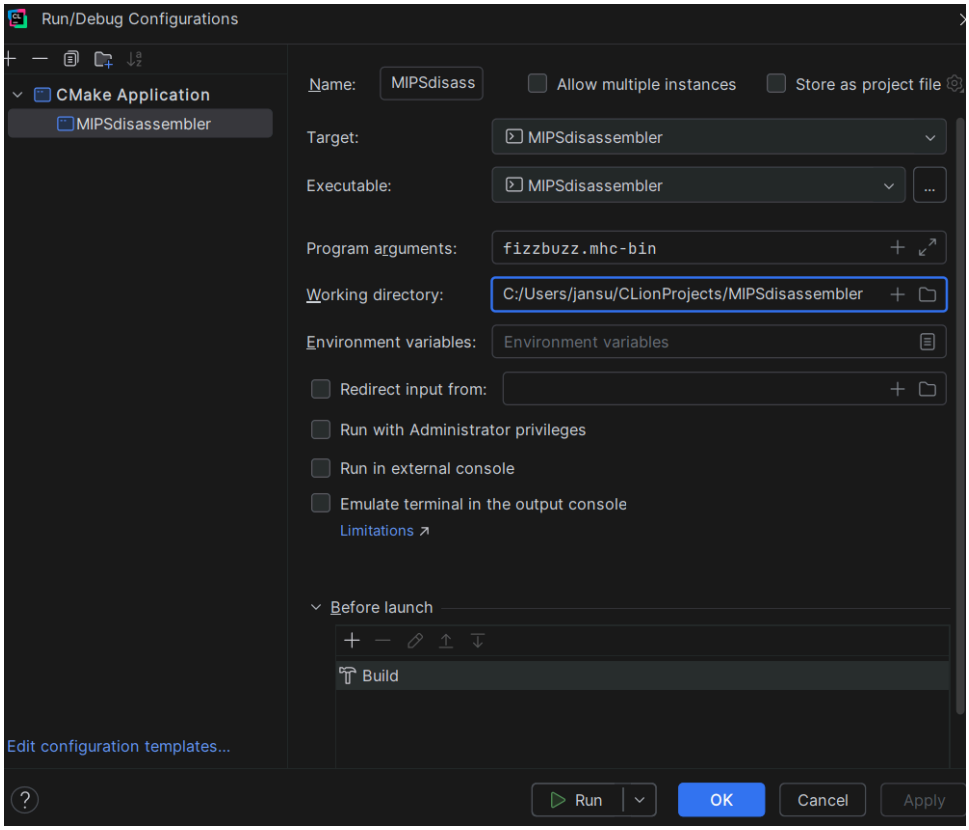
In -ascii, it will display the machine code with each 0 or 1 being a full byte when seen in text in a program like notepad. Each line of could would end up being 32 bytes compared to the original 4 in assembly. In -bin, it is actually saved in binary with each 0 or 1 being a bit. The data stays the same size of 4 bytes, but when seen in text such as notepad, it may look corrupted/strange as it attempts to display in ascii format.

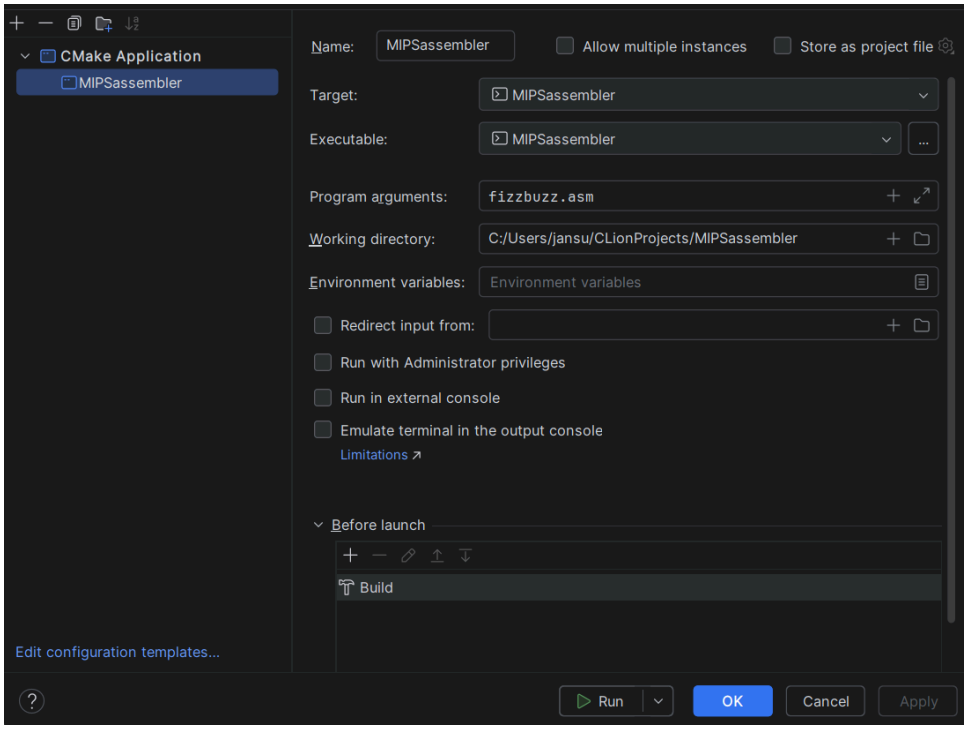
ALL FILES:
https://github.com/jansunx2/MIPSAssembler
https://github.com/jansunx2/MIPSdisassembler
Heavy use of AI

In the assembler project, files with FIRST at the end of the file name are part of the first set of files made/used with the original fizzbuzz file while files without were made/used using the reconstructed version by the disassembler. fizzbuzz.asm = reconstructed.asm

Comparing the output of .mhc-ascii for both original and reconstructed fizzbuzz files (I think exactly the same?)







Used configuration to run ^^^